

# Discrete Mathematics

## Week 1 · Logic & Propositions

JeYoung Park

SeoulTech CS&CE

2026-03-20, 18:00 – 20:00

# WHY

## ① CS/CE 심화 과목의 뼈대 (Gateway)

자료구조, 알고리즘, DB, 오토마타 등을 이해하기 위한 **필수 언어**입니다.

## ② 수학적 성숙도 (Mathematical Maturity)

단순 계산이 아닌, 논리적 주장을 **이해하고 직접 만들어내는** 능력을 기릅니다.

## ③ 상황 해석

정해진 풀이법 암기가 아닌, **창의적으로 문제를 뚫어내는** 사고력.

# Proposition & Logical Operators

## Proposition (명제)

참(T) 또는 거짓(F)으로 **명확히 판별**할 수 있는 문장.

### 논리 연산자 (Logical Operators):

| 기호                | 이름                   | 설명 / C++                           |
|-------------------|----------------------|------------------------------------|
| $\wedge$          | Conjunction (AND)    | 둘 다 참일 때 참 <code>&amp;&amp;</code> |
| $\vee$            | Disjunction (OR)     | 하나라도 참이면 참 <code>  </code>         |
| $\neg$            | Negation (NOT)       | 참/거짓 반전 <code>!</code>             |
| $\oplus$          | Exclusive OR (XOR)   | 진리값이 다를 때 참 <code>^</code>         |
| $\rightarrow$     | Implication (조건문)    | p이면 q이다                            |
| $\leftrightarrow$ | Biconditional (쌍조건문) | 진리값이 같을 때 참                        |

# Truth Table

## 주요 연산자 진리표

| p | q | $p \wedge q$ | $p \vee q$ |
|---|---|--------------|------------|
| T | T | T            | T          |
| T | F | F            | T          |
| F | T | F            | T          |
| F | F | F            | F          |

## $p \rightarrow q$ 진리표

| p (가정) | q (결론) | $p \rightarrow q$ |
|--------|--------|-------------------|
| T      | T      | T                 |
| T      | F      | F                 |
| F      | T      | T★                |
| F      | F      | T★                |

$p \rightarrow q$  가 F가 되는 유일한 경우

$p = T, q = F$  (반례 존재)

★ 공허한 참 (Vacuous Truth)

가정  $p = F$  이면,  $q$  에 무관하게 항상 T

## REMARK

$\rightarrow$  는 AND / OR 과 달리 직관에 어긋나는 결과가 있습니다. 다음 슬라이드에서 증명합니다.

# Vacuous Truth

## 정의

$p \rightarrow q$  에서 가정  $p$ 가 거짓(F)일 때, 결론  $q$ 에 무관하게 전체 명제가 참(T)이 되는 현상.

## 증명 1 — 논리적 동치 (Equivalence) 로 증명

$$p \rightarrow q \equiv \neg p \vee q \quad p = F \Rightarrow \neg p = T \Rightarrow T \vee q = T \quad (q \text{ 값과 완전히 무관})$$

## 증명 2 — 반례 (Counterexample) 의 부재

$p \rightarrow q$  가 거짓이 되는 유일한 경우: “ $p = T$  인데  $q = F$ ” 뿐입니다.  
 $p$ 가 거짓이라면 이 반례를 구성할 수단 자체가 없으므로, 수학적으로 참으로 인정합니다.

## 공허한 참 ( $F \rightarrow T$ )

$$F \rightarrow T = T \quad \checkmark$$

## 유일한 거짓 ( $T \rightarrow F$ )

$$T \rightarrow F = F \quad \times$$

# Vacuous Truth (Cont.)

## 한정자와 빈 집합

$\forall x \in \emptyset, P(x)$  는 명백한 공허한 참 — 반례를 찾을 수단 자체가 없으므로 항상 T

```
#include <algorithm>
#include <vector>
std::vector<int> empty_vec; // empty array (no elements)
// "Every element of this array is even."
// No counterexample can be found -> Vacuously True
bool result = std::all_of(
    empty_vec.begin(), empty_vec.end(),
    [](int x) { return x % 2 == 0; }
);
// result == true <- Vacuous Truth in action
```

## REMARK

빈 배열이나 Null 상태의 데이터가 입력됐을 때, Validation 루프가 공허한 참으로 인해 true를 반환하여 보안·접근 제어가 **뚫리는 버그**가 자주 발생합니다.

# Tautology · Contradiction · Contingency

## $\top$ Tautology

(항진명제) — 항상  $\top$

예:  $p \vee \neg p$

$p \vee \neg p$  진리표

| p | $\neg p$ | 결과 |
|---|----------|----|
| T | F        | T  |
| F | T        | T  |

## $\perp$ Contradiction

(모순명제) — 항상  $F$

예:  $p \wedge \neg p$

$p \wedge \neg p$  진리표

| p | $\neg p$ | 결과 |
|---|----------|----|
| T | F        | F  |
| F | T        | F  |

## $\sim$ Contingency

(우연명제) — T/F가 달라짐

예:  $p \vee q$

$p \vee q$  진리표

| p | q | 결과 |
|---|---|----|
| T | T | T  |
| T | F | T  |
| F | T | T  |
| F | F | F  |

## REMARK

**Tautology:** 논리 동치 법칙의 근거 (법칙 자체가 항진명제)

**Contradiction:** SAT(충족 가능성 문제)에서 불가능성을 판단하는 기준

# Satisfiable · Contingency · Tautology (Cont.)

- **Satisfiable**: 참이 되는 조합이 존재
- **Tautology**: 항상 T ( $\subseteq$  Satisfiable)

- **Contingency**: SAT이지만 Tautology는 아님
- **Contradiction**: 참인 조합 없음 (= Unsatisfiable)

포함 관계 (Venn Diagram)



| 분류            | T  | F  | SAT? |
|---------------|----|----|------|
| Tautology     | 항상 | 없음 | Yes  |
| Contingency   | 있음 | 있음 | Yes  |
| Contradiction | 없음 | 항상 | No   |

**Satisfiable** = Tautology  $\cup$  Contingency  
**Unsatisfiable** = Contradiction



# Logical Equivalences

복잡한 명제(조건문)를 수학적으로 **Refactoring**하기 위한 핵심 법칙들입니다.

| 동치 법칙 (Equivalence)                                                                       | 이름 (Name)               |
|-------------------------------------------------------------------------------------------|-------------------------|
| $p \wedge \mathbf{T} \equiv p \quad / \quad p \vee \mathbf{F} \equiv p$                   | Identity laws           |
| $p \vee \mathbf{T} \equiv \mathbf{T} \quad / \quad p \wedge \mathbf{F} \equiv \mathbf{F}$ | Domination laws         |
| $p \vee p \equiv p \quad / \quad p \wedge p \equiv p$                                     | Idempotent laws         |
| $\neg(\neg p) \equiv p$                                                                   | Double negation law     |
| $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$                                 | Distributive laws       |
| $\neg(p \wedge q) \equiv \neg p \vee \neg q$                                              | De Morgan's laws        |
| $p \vee (p \wedge q) \equiv p \quad / \quad p \wedge (p \vee q) \equiv p$                 | Absorption laws         |
| $p \rightarrow q \equiv \neg p \vee q$                                                    | <b>Conditional ★ 핵심</b> |

# Proposition으로 변환

요구사항 (변수: A=Admin, C=Active, P=Premium)

“유저가 관리자이면서 활성 상태이거나, 관리자는 아니지만 활성 상태이고 프리미엄이거나, 활성 상태이면서 프리미엄이 아닌 경우에 한하여 접근을 허용하라.”

문장 → 기호 대응:

“관리자이면서 활성”

$\Rightarrow A \wedge C$

“관리자는 아니지만 활성 + 프리미엄”

$\Rightarrow \neg A \wedge C \wedge P$

“활성이면서 프리미엄이 아님”

$\Rightarrow C \wedge \neg P$

세 조건 중 하나라도 만족

$\Rightarrow \vee$  로 연결

직역 Proposition:

$$(A \wedge C) \vee (\neg A \wedge C \wedge P) \vee (C \wedge \neg P)$$

```
if ((isAdmin && isActive) ||
    (!isAdmin && isActive && isPremium) ||
    (isActive && !isPremium)) { grantAccess(); }
```

⇒ 질문

Logical Equivalences를 사용하면 이 식을 더 간단하게 줄일 수 있을까요?

# Proposition Reduction

주어진 명제:  $(A \wedge C) \vee (\neg A \wedge C \wedge P) \vee (C \wedge \neg P)$

$$\equiv C \wedge (A \vee (\neg A \wedge P) \vee \neg P)$$

$$\equiv C \wedge (A \vee P \vee \neg P)$$

$$\equiv C \wedge (A \vee \mathbf{T})$$

$$\equiv C \wedge \mathbf{T}$$

$$\equiv \boxed{C}$$

Distributive law (C로 묶기)

Absorption:  $X \vee (\neg X \wedge Y) \equiv X \vee Y$

Negation:  $P \vee \neg P \equiv \mathbf{T}$

Domination:  $A \vee \mathbf{T} \equiv \mathbf{T}$

Identity:  $C \wedge \mathbf{T} \equiv C$  ✓ 완성!

## 증명 결과

복잡했던 기획서의 진짜 의도는 단 하나였습니다.

“활성 상태(C)인 유저에게 접근을 허용하라.”

# Refactoring

## BEFORE

```
// Hard to read, bug-prone
if ((isAdmin && isActive) ||
    (!isAdmin && isActive
     && isPremium) ||
    (isActive && !isPremium))
```

## AFTER

```
// Proven equivalent
if (isActive) {
    grantAccess();
}
```

## REMARK

- 복잡한 다변수 조건문도 논리 연산을 통해 **완전한 Reduction**이 가능합니다.
- 이산수학은 단순한 이론이 아닌, 코드를 **논리적으로 완벽하게** 만드는 도구입니다.

### Before

조건 3개 · 변수 참조 7회 · 높은 복잡도

### After

조건 1개 · 변수 참조 1회 · 최소 복잡도

# Predicate & Quantifier

## REMARK

지금까지는 **고정된 값**의 참/거짓을 다뤘습니다. 실제 코드는 **변수**를 다룹니다. → Predicate(술어)가 필요합니다.

## Predicate (술어)란?

주어의 성질이나 관계를 나타내는 말. 명제 함수  $P(x)$ 는 변수가 채워져야 참/거짓 판별 가능.

### $\forall x P(x)$ — Universal

모든  $x$ 에 대해 참. 반례 하나라도 있으면 거짓.

```
// All elements even?  
std::all_of(v.begin(), v.end(),  
    [](int x){ return x%2==0; });  
// empty vec -> Vacuously True!
```

### $\exists x P(x)$ — Existential

조건을 만족하는  $x$ 가 하나라도 존재.

```
// Any even element?  
std::any_of(v.begin(), v.end(),  
    [](int x){ return x%2==0; });  
// empty vec -> always false
```

## 부정 (Negation)

$$\neg(\forall x P(x)) \equiv \exists x \neg P(x)$$

$$\neg(\exists x P(x)) \equiv \forall x \neg P(x)$$

# Satisfiability (SAT)

## Satisfiability 란?

주어진 복합 명제가 참(True)이 되게 만드는 변수값 조합(Truth Assignment)이 적어도 하나 존재하는가?

### Satisfiable

$$p \vee \neg q$$

참이 되는 조합 존재.

예:  $p=T$  이면 참.

### Contradiction

$$p \wedge \neg p$$

참이 되는 조합 없음.

= Unsatisfiable

### Tautology

$$p \vee \neg p$$

항상 참.

Satisfiable의 특수 경우

## REMARK

현실의 복잡한 제약 조건 문제(배치, 스케줄링, 회로 설계 등)를 명제 논리식으로 변환하면, 그 식이 **Satisfiable** 한지 판단하는 것 = 제약 조건을 만족하는 해(**Solution**)를 찾는 것입니다.

→ 지금부터 n-Queens 문제를 SAT 로 직접 모델링해봅시다!

# n-Queens

## 문제 (Problem Statement)

$n \times n$  체스판에  $n$ 개의 퀸(Queen)을 배치하되, 어떤 두 퀸도 서로 공격하지 못하게 할 수 있는가?  
(퀸은 같은 행·열·대각선 위의 모든 칸을 공격합니다.)

### 4-Queens 해답 예시

|   |   |   |   |
|---|---|---|---|
|   | Q |   |   |
|   |   |   | Q |
| Q |   |   |   |
|   |   | Q |   |

행·열·대각선 모두 충돌 없음

### SAT 변수 정의

$p(i, j)$  = “ $i$ 행  $j$ 열에 퀸이 있다”

**True:** 해당 칸에 퀸 있음

**False:** 해당 칸에 퀸 없음

$n \times n$  개의 불리언 변수  $p(1, 1), p(1, 2), \dots, p(n, n)$

### 우리의 목표

다음 5가지 제약 조건을 모두 만족하는

$p(i, j)$  값의 조합을 찾아라!

⇒ 이것이 바로 SAT 문제입니다.

# n-Queens 모델링: $Q_1$

조건  $Q_1$ : 각 행(Row)에는 퀸이 **최소 1개** 있어야 한다

$$Q_1 = \bigwedge_{i=1}^n \left( \bigvee_{j=1}^n p(i, j) \right)$$

REMARK:  $\bigwedge$  = 큰 AND,  $\bigvee$  = 큰 OR — 여러 항을 한꺼번에 묶는 표기법입니다.

**직관적으로 이해하기:**

- $\bigvee_{j=1}^n p(i, j)$ : “ $i$ 번 행의 어떤 열에든 **하나라도** 퀸이 있다”  
 $= p(i, 1) \vee p(i, 2) \vee \cdots \vee p(i, n)$
- $\bigwedge_{i=1}^n$ : 위 조건을 **모든 행(1행부터  $n$ 행까지)**에 동시에 요구한다

## 4-Queens 예시 ( $n = 4$ )

$$\begin{aligned} Q_1 = & (p(1, 1) \vee p(1, 2) \vee p(1, 3) \vee p(1, 4)) \\ & \wedge (p(2, 1) \vee p(2, 2) \vee p(2, 3) \vee p(2, 4)) \\ & \wedge (p(3, 1) \vee p(3, 2) \vee p(3, 3) \vee p(3, 4)) \\ & \wedge (p(4, 1) \vee p(4, 2) \vee p(4, 3) \vee p(4, 4)) \end{aligned}$$



# n-Queens 모델링: $Q_2$

조건  $Q_2$ : 각 행(Row)에는 퀸이 **최대 1개**여야 한다

$$Q_2 = \bigwedge_{i=1}^n \bigwedge_{j=1}^{n-1} \bigwedge_{k=j+1}^n (\neg p(i, j) \vee \neg p(i, k))$$

왜 이 식이 “최대 1개”를 표현할까? — 단계별 논리 전개

- 1 목표: 같은 행  $i$ 에서, 임의의 두 열  $j \neq k$ 에 퀸이 **동시에 있으면 안 됨**
- 2 금지 조건:  $p(i, j) \wedge p(i, k) = \text{False}$
- 3 양쪽에  $\neg$ 를 씌우기:  $\neg(p(i, j) \wedge p(i, k)) = \text{True}$
- 4 De Morgan 적용:  $\neg p(i, j) \vee \neg p(i, k) = \text{True}$   
“두 칸 중 적어도 하나는 반드시 비어 있어야 한다”
- 5 모든 쌍·모든 행에 적용:  $j < k$ 인 모든 쌍, 모든 행  $i$ 에 대해  $\wedge$ 으로 묶기

$Q_1 \wedge Q_2 =$  각 행에 **정확히 1개**의 퀸이 존재

# n-Queens 모델링: $Q_3$

조건  $Q_3$ : 각 열(Column)에는 퀸이 최대 1개여야 한다

$$Q_3 = \bigwedge_{j=1}^n \bigwedge_{i=1}^{n-1} \bigwedge_{k=i+1}^n (\neg p(i,j) \vee \neg p(k,j))$$

$Q_2$ 와 완전히 같은 논리 — 행과 열만 바뀐 것!

## $Q_2$ 와 대응 비교

|    | $Q_2$ (행 제약)                   | $Q_3$ (열 제약)                   |
|----|--------------------------------|--------------------------------|
| 고정 | 행 $i$ 고정                       | 열 $j$ 고정                       |
| 비교 | 열 $j \neq k$ 비교                | 행 $i \neq k$ 비교                |
| 금지 | $\neg p(i,j) \vee \neg p(i,k)$ | $\neg p(i,j) \vee \neg p(k,j)$ |
| 의미 | 같은 행에 두 퀸 금지                   | 같은 열에 두 퀸 금지                   |

## 직관 + 요약

같은 열에서 서로 다른 두 행을 뽑으면 둘 다 퀸이면 안 됩니다.  $\rightarrow$  De Morgan:  $\neg p(i,j) \vee \neg p(k,j)$

$Q_1 \wedge Q_2 \wedge Q_3$  완료  $\Rightarrow$  행·열 제약 달성. 남은 것: 대각선!

# n-Queens 모델링: $Q_4, Q_5$

$Q_4$ : ↗ 방향 대각선 — 최대 1개

$$Q_4 = \bigwedge_{i,j,k \geq 1} (\neg p(i,j) \vee \neg p(i-k, j+k))$$

(i,j)에서 위·오른쪽으로 k칸  $\Rightarrow$  같은 ↗ 대각선  
두 칸에 동시에 퀸 금지  $\rightarrow$  De Morgan 적용

$Q_5$ : ↘ 방향 대각선 — 최대 1개

$$Q_5 = \bigwedge_{i,j,k \geq 1} (\neg p(i,j) \vee \neg p(i+k, j+k))$$

(i,j)에서 아래·오른쪽으로 k칸  $\Rightarrow$  같은 ↘ 대각선  
두 칸에 동시에 퀸 금지  $\rightarrow$  De Morgan 적용

최종 SAT Equation

$$Q = Q_1 \wedge Q_2 \wedge Q_3 \wedge Q_4 \wedge Q_5$$

이 식을 **True**로 만드는  $p(i,j)$  조합 = n-Queens  
정답

같은 색 = 같은 ↘ 대각선

|       |       |       |       |
|-------|-------|-------|-------|
| (1,1) | (1,2) | (1,3) | (1,4) |
| (2,1) | (2,2) | (2,3) | (2,4) |
| (3,1) | (3,2) | (3,3) | (3,4) |
| (4,1) | (4,2) | (4,3) | (4,4) |

(i,j)와 (i',j')가 같은 ↘ 대각선

$$\Leftrightarrow i' - i = j' - j$$

$$\Leftrightarrow (i', j') = (i+k, j+k)$$

↗ 대각선은?

$$\Leftrightarrow i' - i = -(j' - j)$$

$$\Leftrightarrow (i', j') = (i-k, j+k)$$

## Silver BOJ 11723 (집합)

**목표:**  $\wedge, \vee, \neg, \oplus$  논리 기호를 비트 연산으로 직역하기

**힌트:** 비트마스크에서 원소 유무(비트 1/0)가 논리 기호와 1:1 대응됩니다.

## Gold BOJ 1987 (알파벳)

**목표:**  $\exists$  판별과 비트마스크 방문 처리 최적화

**힌트:** 방문한 알파벳 집합을 비트 연산으로 관리하면 메모리 효율이 대폭 향상됩니다.

## Gold BOJ 14257 (XOR 합)

**목표:**  $A + B = S$ 와  $A \oplus B = X$ 의 논리적 관계 분석

**힌트:**  $A + B = (A \oplus B) + 2(A \wedge B)$  항등식을 활용하여 Carry 비트를 추적하세요.

# To be continue...

## Week 1 — 오늘

참/거짓을 다루는  
명제와 논리의 언어

$$P \wedge Q \quad P \vee Q \quad P \rightarrow Q$$

Proposition · Logic · SAT

## Week 2 — 다음 주

참인 것들을 구조적으로  
다루는 언어

$$P \wedge Q \leftrightarrow A \cap B$$

$$P \vee Q \leftrightarrow A \cup B$$

Sets · Matrix Operations

오늘 배운 논리 연산을 확장하여, 수많은 오브젝트의 상태를 한꺼번에 처리하는  
**집합(Sets)과 행렬(Matrix)**으로 넘어갈 예정입니다.